NAVAJA — Log de sesión
Fecha: 15 de marzo 2026
Proyecto: SaaS de turnos para barberías

═══

CONTEXTO DEL PROYECTO

═══

Navaja es un SaaS de gestión de turnos para barberías.
Repo: https://github.com/credens/navaja
Stack: Next.js 14, Supabase (PostgreSQL), MercadoPago, Meta Cloud API (WhatsApp)
Deploy local mientras desarrolla, luego Vercel.

═══

DECISIONES DE DISEÑO Y NEGOCIO

═══

MODELO DE NEGOCIO:
- La plataforma (Navaja) cobra al cliente y liquida a la barbería
- El cliente paga al reservar (online, obligatorio)
- Comisión configurable por barbería
- Canon mensual a cada barbería (suscripción MP)
- Trial de 7 días gratis al conectar MercadoPago

PLANES:
- Base: precio fijo mensual, hasta 150 turnos, hasta 3 barberos
- Pro: precio base + cobro por turno sobre 150, barberos ilimitados

POLÍTICA DE CANCELACIÓN:
- Cancelación gratuita hasta 3 horas antes del turno
- Cancelación tardía → cobra igual (sin reembolso)
- Cancelación por el local → reembolso total siempre
- No-show → cobra igual (configurable por barbería)

BARBEROS:
- No tienen panel propio (se decide en fase 2)
- Reciben notificaciones por WhatsApp
- La barbería maneja los turnos

NOTIFICACIONES:
- Todo por WhatsApp (Meta Cloud API)
- Sin email
- La barbería puede usar número propio (modo B) o el de la plataforma (modo A)
- Cliente puede cancelar respondiendo CANCELAR al WhatsApp

MERCADOPAGO:
- MP Marketplace para split automático plataforma → barbería
- Checkout Pro para pago del cliente

```
- Suscripciones para canon mensual
- OAuth para vincular cuenta MP de cada barbería

═══════════════════════════════════════════════════════════════
## ARQUITECTURA
═══════════════════════════════════════════════════════════════

ESTRUCTURA DE CARPETAS:
navaja/
├── app/
|   ├── page.tsx                  ✅ Landing page
|   ├── layout.tsx                ✅ Root layout
|   ├── globals.css               ✅ CSS global
|   ├── icon.tsx                  ✅ Favicon
|   ├── _components/
|   |   └── LandingScripts.tsx    ✅ Scripts de scroll (client component)
|   ├── [slug]/                   ✅ Turnera pública por barbería
|   |   ├── page.tsx              ✅ Server component con datos
|   |   ├── loading.tsx           ✅ Skeleton
|   |   └── _components/
|   |       ├── reserva.css       ✅ CSS mobile
|   |       ├── ReservaFlow.tsx   ✅ Orquestador del flujo
|   |       ├── StepServicio.tsx  ✅ Paso 1
|   |       ├── StepBarbero.tsx   ✅ Paso 2
|   |       ├── StepFecha.tsx     ✅ Paso 3 (calendario + slots)
|   |       ├── StepDatos.tsx     ✅ Paso 4 (datos del cliente)
|   |       ├── StepResumen.tsx   ✅ Paso 5 (resumen + pago MP)
|   |       └── StepConfirmacion.tsx ✅ Paso 6
|   ├── api/
|   |   ├── turnos/
|   |   |   ├── route.ts          ✅ POST crear turno
|   |   |   ├── [id]/route.ts     ✅ PATCH cancelar
|   |   |   └── slots/route.ts    ✅ GET slots disponibles
|   |   ├── mp/
|   |   |   └── callback/route.ts ✅ OAuth MP
|   |   ├── webhooks/
|   |   |   ├── mp/route.ts       ✅ Webhook pagos MP
|   |   |   └── whatsapp/route.ts ✅ Webhook WSP entrante
|   |   └── cron/
|   |       ├── turnos/route.ts   ✅ No-shows + recordatorios (cada 15min)
|   |       └── suscripciones/route.ts ✅ Trials + canon (diario)
|   ├── dashboard/                ⬜ PENDIENTE
|   └── registro/                 ⬜ PENDIENTE
├── lib/
|   ├── supabase/
|   |   ├── server.ts             ✅ Cliente SSR + Admin
|   |   └── client.ts             ✅ Cliente browser
|   ├── services/
```

```
97  |   |   ├── agenda.ts              ✅ Cálculo de slots
98  |   |   ├── comisiones.ts          ✅ Split de pagos
99  |   |   ├── cancelaciones.ts       ✅ Regla de 3hs
100 |   |   ├── barberos.ts            ✅ Alta y baja
101 |   |   └── liquidaciones.ts       ✅ Generación de períodos
102 |   ├── mp/
103 |   |   ├── client.ts              ✅ Preferencias y OAuth
104 |   |   └── refunds.ts             ✅ Reembolsos
105 |   └── whatsapp/
106 |       ├── client.ts              ✅ Templates y mensajes
107 |       └── onboarding.ts          ✅ Notificaciones de registro
108 ├── types/
109 |   └── index.ts                   ✅ Tipos TypeScript
110 ├── supabase/
111 |   ├── config.toml                ✅ Config local
112 |   └── migrations/
113 |       └── 20260315000000_initial.sql ✅ Schema completo
114 ├── vercel.json                    ✅ Cron jobs config
115 ├── package.json                   ✅
116 ├── tsconfig.json                  ✅
117 └── .env.example                   ✅
118
119 ═══════════════════════════════════════════════════════════
120 ## BASE DE DATOS
121 ═══════════════════════════════════════════════════════════
122
123 TABLAS CREADAS (19 total):
124 - plataforma_config
125 - planes
126 - barberias
127 - barberias_horarios
128 - usuarios
129 - barberos
130 - barberos_disponibilidad
131 - barberos_bloqueos
132 - servicios
133 - barberos_servicios
134 - clientes
135 - turnos
136 - liquidaciones
137 - liquidaciones_detalle
138 - bajas_pendientes
139 - notificaciones_log
140 - reembolsos_pendientes
141 - barberias_planes_historial
142 - usuarios_pendientes
143
144 TRIGGERS:
```

- set_actualizado_en → barberias, turnos
- calcular_fecha_hora_fin → turnos (calcula fin desde duración del servicio)
- snapshot_comisiones → turnos (calcula el split al insertar)

VISTAS:
- v_turnos_completo
- v_agenda_dia
- v_liquidaciones_pendientes
- v_metricas_barberia

SEED (datos de prueba):
- Barbería: Barber Kings (slug: barber-kings)
- 3 barberos: Martín (50%), Lucas (45%), Sebastián (40%)
- 4 servicios: Corte $8k, Corte+Barba $12k, Corte+Barba+Lavado $15k, Arreglo barba $6k
- Horarios: lun-vie 9-20hs, sáb 9-18hs

═══
LÓGICA DE NEGOCIO IMPLEMENTADA
═══

1. CÁLCULO DE SLOTS (lib/services/agenda.ts)
   - Grilla de 15 min independiente de la duración del servicio
   - Intersección horario barbero ∩ horario barbería
   - Verifica bloqueos y turnos existentes
   - Descarta slots pasados (15 min de buffer)

2. CÁLCULO DE COMISIONES (lib/services/comisiones.ts)
   - MP (~5.99%) → monto_neto
   - Plataforma (% configurable) → monto_plataforma
   - Barbero (% configurable) → monto_barbero
   - Dueño absorbe centavos de redondeo

3. CANCELACIONES (lib/services/cancelaciones.ts)
   - Regla de 3hs: antes = reembolso, después = cobra igual
   - Local cancela → siempre reembolso
   - Reembolsos fallidos → tabla reembolsos_pendientes para reintentar

4. ALTA/BAJA BARBEROS (lib/services/barberos.ts)
   - Alta: crea usuario + barbero + disponibilidad + servicios
   - Baja: bloquea nuevas reservas → cancela turnos futuros → genera liquidación de cierre → da de baja
   - Baja pendiente: si hay turno en curso, espera que termine

5. LIQUIDACIONES (lib/services/liquidaciones.ts)
   - Incluye: completado, cancelado_sin_reembolso, no_show
   - Siempre genera liquidación aunque sea $0 (registro de cierre)
   - Detalle por turno

```
═══════════════════════════════════════════════════════════
## ESTADOS DE UN TURNO
═══════════════════════════════════════════════════════════

pendiente_pago → confirmado → en_curso → completado
                    ↓
              cancelado              (a tiempo, reembolso)
              cancelado_sin_reembolso (tardío, cobra igual)
              cancelado_por_local    (local cancela, reembolso)
              no_show                (no se presentó, cobra igual)

═══════════════════════════════════════════════════════════
## WHATSAPP TEMPLATES (a aprobar en Meta)
═══════════════════════════════════════════════════════════

- turno_confirmado
- turno_recordatorio_24hs
- turno_recordatorio_2hs
- turno_cancelado_reembolso
- turno_cancelado_sin_reembolso
- turno_cancelado_por_local
- aviso_barbero_turno_nuevo
- aviso_barbero_turno_cancelado
- aviso_no_show
- liquidacion_generada
- liquidacion_pagada
- canon_fallido
- onboarding_mp_conectado
- trial_por_vencer
- trial_vencido

═══════════════════════════════════════════════════════════
## SETUP LOCAL
═══════════════════════════════════════════════════════════

PREREQUISITOS INSTALADOS:
- Node.js v24
- Git
- Docker Desktop
- Supabase CLI (via brew)
- Homebrew

COMANDOS:
  npm run dev          → Next.js en localhost:3000
  supabase start       → Arranca Supabase (Docker)
  supabase stop        → Detiene Supabase
  supabase db reset    → Resetea DB y corre migraciones
  supabase status      → Ver URLs y keys
```

```
URLS LOCALES:
  App:    http://localhost:3000
  Studio: http://localhost:54323
  API:    http://127.0.0.1:54321

.env.local (variables necesarias):
  NEXT_PUBLIC_SUPABASE_URL=http://127.0.0.1:54321
  NEXT_PUBLIC_SUPABASE_ANON_KEY=sb_publishable_...
  SUPABASE_SERVICE_ROLE_KEY=sb_secret_...
  NEXT_PUBLIC_APP_URL=http://localhost:3000

NOTA IMPORTANTE — Keys de Supabase local:
  Las keys se obtienen corriendo: supabase start (o supabase stop && supabase start)
  Aparecen en la sección "Authentication" como Publishable y Secret
  Publishable → NEXT_PUBLIC_SUPABASE_ANON_KEY
  Secret      → SUPABASE_SERVICE_ROLE_KEY
  Ambas empiezan con "sb_"

NOTA IMPORTANTE — URL de Supabase:
  La URL correcta es http://127.0.0.1:54321 (sin /storage/v1/s3 al final)

═══════════════════════════════════════════════════════════
## PÁGINAS COMPLETADAS
═══════════════════════════════════════════════════════════

✅ Landing page          → http://localhost:3000
✅ Flujo reserva cliente → http://localhost:3000/barber-kings

═══════════════════════════════════════════════════════════
## PENDIENTE PARA PRÓXIMA SESIÓN
═══════════════════════════════════════════════════════════

⬜ Dashboard del dueño   → /dashboard
⬜ Registro de barbería  → /registro
⬜ Push a GitHub         → credens/navaja
⬜ Integración MP real (sandbox)
⬜ Integración WhatsApp (Meta sandbox)
⬜ Panel del barbero     → fase 2

═══════════════════════════════════════════════════════════
## PROBLEMAS RESUELTOS EN ESTA SESIÓN
═══════════════════════════════════════════════════════════

1. Supabase CLI no se instala con npm → usar brew install supabase/tap/supabase
2. UUIDs inválidos en seed (s1000000...) → corregidos a a0000001...
3. Error de hidratación en Next.js → mover CSS a globals.css + LandingScripts client component
4. SUPABASE_URL con /storage/v1/s3 → usar solo http://127.0.0.1:54321
```

5. JWT inválido ("Expected 3 parts") → keys mal copiadas del Studio, usar las de supabase start
6. RLS bloqueando queries → usar createAdminClient en page.tsx del slug
7. Carpeta [slug] con backslash → usar comillas simples en terminal Mac